Application of Virtualization and Containerization Technologies in IP Telephony

V. M. Antonova
Moscow Technical University of Communications and Informatics
Moscow, Russia
V.A. Kotelnikov Institute of Radio Engineering and Electronics of the Russian Academy of Sciences
Moscow, Russia
v.m.antonova@mtuci.ru

E. E. Malikova
Moscow Technical University of Communications and Informatics
Moscow, Russia
e.e.malikova@mtuci.ru

A. Y. Malikov
Moscow Technical University of Communications and Informatics
Moscow, Russia

M. A. Fartyshev
Moscow Technical University of Communications and Informatics
Moscow, Russia

2025 Systems of Signals Generating and Processing in the Field of on Board Communications | 979-8-3315-3263-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/IEEECONF64229.2025.10948100

***Abstract*— The article considers the contemporary trends that telecommunication companies are facing in the rapidly changing technological environment. One of the trends is the technology of network virtualization, which allows to significantly increase the efficiency of network resources through the organization of multiple virtual resources based on one physical resource. The objective of this study is to analyze new tools available for the implementation of a virtual IP PBX. Virtualization and containerization technologies that affect the efficient operation of VoIP services are studied in the paper. The implementation of these technologies has been shown to facilitate more efficient use of server resources and more secure interaction with multiple tools for analyzing system operation. The principles of integrating software telephone exchanges into the virtualized environment are considered. Special attention is paid to the integration of a virtual Asterisk exchange into the containerized environment. Approaches to automating Asterisk's deployment and management in the microservice architecture using a Kubernetes platform are also considered. It is recommended that virtualization and containerization technologies should be incorporated into the curricula of technical universities.)**

Keywords—virtualization; container; container orchestration; IP telephony; VoIP; Kubernetes Cluster; K8s; Asterisk;Kamailio

I. INTRODUCTION

Voice transmission through packet switched networks over the Internet Protocol (VoIP – Voice over IP) is currently a widespread technology across telecommunication networks [1, 2]. Readily available solutions based on the equipment of prominent companies such as Cisco, Avaya, Iskratel and others can be used for IP telephony. IP PBXs built on software solutions are also widely accepted in contemporary networks. These IP PBXs play an important role in the network infrastructure, offering flexibility, scalability and economic efficiency of the exchange.

IP PBXs such as Asterisk, Freeswitch and Kamailio are built on the basis of open software systems. Asterisk, an open-source software PBX [3, 4] is a highly popular solution in the field of IP telephony. It is a very flexible platform that is employed to organize communications in various offices and large industrial enterprises. The PBX's functionality makes it possible to realize almost every function, for example, video conferencing, voice recognition function, creation of interactive voice menu (IVR or Interactive Voice Response) and others [5].

Network Functions Virtualization (NFV) is a technology that has had a profound impact on the topologies of telecommunication networks [6, 7]. It enables the abstraction from physical hardware and the creation of multiple isolated virtual environments on a single physical server, thereby providing more efficient use of network resources and simplifying infrastructure management. For instance, 10-20 virtual machines can be run on a single physical server.

The evolution of cloud technologies and containerization has resulted in an increased demand for the efficient deployment and management of the Asterisk software exchange in an environment that provides high fault tolerance, automatic scaling and ease of use. Kubernetes platform (K8s) is a standard for container orchestration that provides all the necessary tools to build a flexible and fault-tolerant IP telephony infrastructure. It is necessary to take into account the interaction of PBX with the SIP signaling protocol and the RTP multimedia traffic transfer protocol [8].

979-8-3315-3263-5/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on December 01,2025 at 14:43:18 UTC from IEEE Xplore. Restrictions apply.

II. Virtualization and Containerization Technologies

Virtualization can be defined as the provision of a set of computational resources or their logical combination abstracted from the hardware implementation while providing logical isolation from each other for the computational processes running on the same physical resource [9].

A virtual machine (VM) is an environment that is presented to the guest operating system as a hardware environment. VM is a very convenient tool for debugging software and conducting experiments without disturbing the main OS, as well as for running several operating systems on one computer concurrently [10]. A hypervisor is responsible for optimal resource allocation between VMs and for isolation of operating systems. Virtualbox is an example of such a hypervisor program.

The following main types of virtualization can currently be distinguished: software and full (hardware) virtualization, operating system (OS) level virtualization and application virtualization.

With *hardware virtualization*, the hypervisor can work directly with physical resources without a host system (Fig. 1). The hypervisor allocates hardware to virtual machines where operating systems are installed. This results in almost full-fledged physical computers. At the same time, virtual machines are completely isolated from each other.

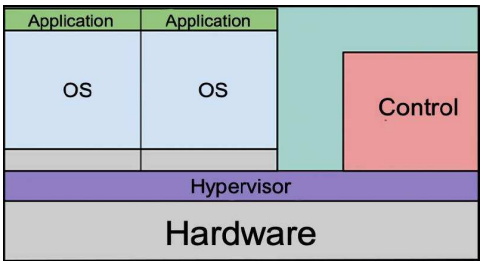

Fig. 1. Full (hardware) virtualization

When using software virtualization, the hypervisor operates in conjunction with physical resources via the host operating system, which is installed on the computer or server (Fig. 2). Software virtualization is used by Oracle VM VirtualBox and VMware Workstation.

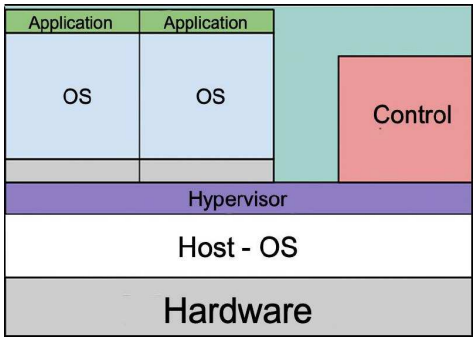

Fig. 2. Software virtualization

In the context of operating system level virtualization, there is no separate hypervisor layer. Instead, the host operating system itself is responsible for the allocation of hardware resources to multiple virtual machines and for supporting their isolation from each other (Fig. 3). These processes are isolated within a separate container. The container, in turn, receives an almost full-fledged operating system where applications can be installed. This type of virtualization has become very popular in recent times.

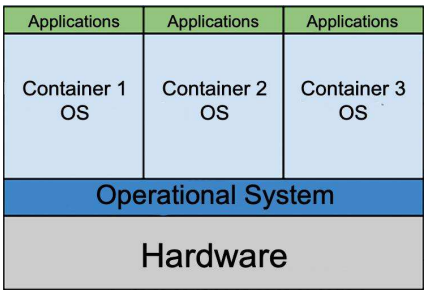

Fig. 3. Operating system level virtualization

Application virtualization enables different applications to run independently of the host operating system. The virtual environment in which a particular application runs separates it from physical resources and the host operating system. This type of virtualization is employed, for example, by Docker [11, 12].

III. Overview of Technologies for Implementing a Virtual PBX

Containerization technologies have evolved, and this has led to an increased demand for efficient deployment and management of IP-PBXs in an environment that provides high fault tolerance, automatic scaling and ease of operation is increasing. The following discussion will consider in more detail the technologies required to deploy Asterisk in the Kubernetes environment.

Kubernetes (K8s) is a platform for automating the deployment, scaling, and management of containerized applications [13]. It is a container orchestration standard that provides all the necessary tools to build a flexible and fault-tolerant IP telephony infrastructure. The platform allows for efficient management of system components, offering load balancing within the cluster, and maintaining sufficient system fault tolerance and automatic service recovery. All settings are described declaratively using manifests in the YAML or JSON format, which is a specific file format perceived by K8s.

The deployment of Asterisk within the Kubernetes environment necessitates the decomposition of a monolithic application into discrete microservices, which allows achieving flexibility, scalability and manageability of the system [14, 15].

The following components constitute the primary elements of the architecture:

1. Asterisk Core, which is the principle service that handles the management of voice calls, the routing of calls, and the processing of media streams. It functions within a

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on December 01,2025 at 14:43:18 UTC from IEEE Xplore. Restrictions apply.

container and necessitates a flexible configuration for interaction with external systems.

2. SIP Proxy (Kamailio), which performs the functions of routing and balancing SIP messages prior to their transmission to Asterisk, thereby reducing the main workload and increasing fault tolerance.

3. Database (PostgreSQL, MySQL), which is used to store configuration, CDR (Call Detail Record) data and user authorization data. PostgreSQL provides reliable data storage and processing, and is easily integrated with Asterisk and other services.

4. Configuration and management (ConfigMaps, Secrets). Kubernetes provides tools for centralized management of configurations and secret data (logins, passwords, encryption keys).

5. Load balancer (Ingress Controller, Load Balancer), which provides traffic distribution between service replicas and ensures an entry point into the cluster.

6. Monitoring and Logging (Prometheus, Grafana), which are used to track metrics, performance and to analyze Asterisk logs.

Asterisk deployment in the Kubernetes cluster can be organized in several ways, each of which has its own features and areas of application.

The monolithic deployment involves running Asterisk in a single container, incorporating all the necessary units. This is the simplest option, suitable for testing or small systems. However, this approach makes it difficult to scale and update the application.

The microservice approach involves splitting Asterisk functions into several containers, including SIP proxy (Kamailio), database (PostgreSQL), RTP proxy (RTP Proxy), etc. This option offers flexibility, fault tolerance and easy upgrades, though it requires more complex customization.

The hybrid approach combines elements of monolithic and microservice deployments, thereby enabling the separation of critical components into discrete services, while leaving the main functions in a single container. This represents a good compromise for a gradual migration.

Practical Implementation of Asterisk IP-PBX in a Kubernetes Cluster

In order to study virtualization technology, students are offered to conduct laboratory experiments on the deployment of an IP-PBX of the Asterisk type in a Kubernetes platform. In this case, it is important to ensure the stable operation of the system when the load on VoIP services increases. As illustrated in Fig. 4, the block diagram shows the interaction of the system components. This diagram illustrates the architecture of the deployed services, including the traffic forwarding scheme. Traffic arrives at the entry point of the Kubernetes cluster when accessing the virtual PBX. The following services are launched as containers: Asterisk, Postgres, Kamailio, and RTPProxy.

The Kubernetes platform is characterized by its own terminology:

Running containers with certain characteristics and applications running in them are called *pods*;

Cluster nodes are both control and compute nodes;

Master Node is responsible for setting tasks accepted from the API server;

Worker Node is responsible for executing the container on its computing resources.

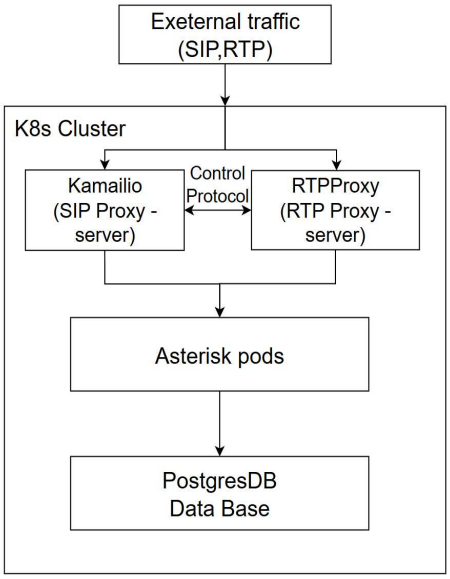

Fig. 4. Block diagram of the system component interaction

The initial step is to deploy a K8s cluster. One of the minimum system configurations involves one Master node and one Worker node. We will use virtual machines with the following specifications:

– Master Node with two virtual processors and 2 GB RAM;

– Worker Node with four virtual processors and 8 GB RAM.

The selection of VM capacities is deliberate as the minimum parameters of a node in a K8s cluster are two CPUs (Central Processing Unit) and 2 GB RAM. For the Master Node this is quite enough, since if there is at least one Worker Node, the containers are started on the Worker Node, while the Master Node is engaged in tracking the viability of pods, restarting, etc.

Subsequently, the K8s cluster must be initialized using the kubeadm tool, for this purpose the following command should be entered on the Master Node:

kubeadm init --apiserver-advertise-address=192.168.255.136

The command will initialize the cluster with the API server address at 192.168.255.136.

In addition, configuration files must be imported into the cluster. It is necessary to deploy a plugin to manage IP addressing of pods, as well as a command to add the first Worker Node to the cluster with the generated token.

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on December 01,2025 at 14:43:18 UTC from IEEE Xplore. Restrictions apply.

Kubernetes tokens are cryptographic credentials that are used to authenticate users and service accounts when interacting with the Kubernetes API server.

The next step is to export the configuration file of the created cluster, in order to interact with it. The subsequent step involves the addition of Worker Nodes to the cluster. The exportation of the configuration file and the generation of a token for the Worker Node connection are illustrated in Fig. 5.

```
root@k8s-master1:/home/k8s-root# export KUBECONFIG=/etc
root@k8s-master1:/home/k8s-root# kubeadm token generate
4p88mq.6lg6gehnzg0mmzkb
root@k8s-master1:/home/k8s-root# kubeadm token create 4
in-command --ttl=0
kubeadm join 192.168.255.136:6443 --token 4p88mq.6lg6ge
ert-hash sha256:de621c473194e05c1256ac622873c30e7ccaccd
root@k8s-master1:/home/k8s-root#
```

Fig. 5. The result of the Worker Node connection

In the subsequent phase, the deployment of Asterisk, Postgres, TRPProxy and Kamailio pods is to be undertaken. The status of all pods in the cluster, including those with the pod IP addressing management plugin, Asterisk, RTPProxy and Kamailio (Fig. 6).

```
root@k8s-master1:/home/k8s-root# kubectl get pod -A -o wide
NAMESPACE     NAME                                        READY
kube-system   cilium-c2clw                                1/1
kube-system   cilium-envoy-5wplh                          1/1
kube-system   cilium-envoy-vzx6m                          1/1
kube-system   cilium-n45nd                                1/1
kube-system   cilium-operator-59f8db7bf5-z89k5            1/1
kube-system   coredns-55cb58b774-9p7ps                    1/1
kube-system   coredns-55cb58b774-bwvxl                    1/1
kube-system   etcd-k8s-master1                            1/1
kube-system   kube-apiserver-k8s-master1                  1/1
kube-system   kube-controller-manager-k8s-master1         1/1
kube-system   kube-proxy-4v844                            1/1
kube-system   kube-proxy-8pgkt                            1/1
kube-system   kube-scheduler-k8s-master1                  1/1
voip          asterisk-0                                  1/1
voip          kamailio-l6m2x                              1/1
voip          postgres-75f7f9c87f-g9j66                   1/1
voip          rtpproxy-65bf8b657-zv5t9                    1/1
```

Fig. 6. Information about running pods

The next step is to initiate a test call in order to make sure that the PBX is functioning correctly.

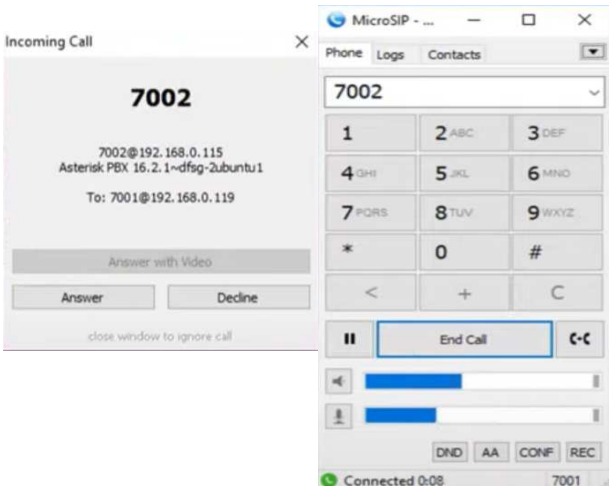

Fig. 7. Positive test call result

In the course of the deployment of Asterisk PBX in the Kubernetes environment, the designated tasks were successfully accomplished. These included the development of the system infrastructure, the automatic deployment and configuration of services. A test call was made. The system demonstrated stable operation, thereby confirming the feasibility of creating highly efficient and scalable solutions for IP telephony in a containerized environment.

IV. Prospects of Using Asterisk in a Kubernetes Cluster

In the process of technology development and increasing requirements for communication services, the Asterisk PBX and Kubernetes platform have good prospects for joint use. At the same time it is important to ensure stable operation of the system when the load on VoIP services increases. Scalability issues are solved by automatically creating additional pods from the Asterisk image. Kubernetes allows you to dynamically scale the system using the Horizontal Pod Autoscaler system. This is a system for scaling the resources available for containers; it monitors the load of CPU and RAM and creates additional pods if necessary. Kamailio in turn plays the role of a balancer, distributing the load of SIP traffic between Asterisk replicas. PostgreSQL database is also scaled by creating replica pods.

The fault tolerance of the system is ensured by mechanisms that automatically restart containers in case of failure. Kubernetes also provides load balancing within the cluster, distributing it evenly across the cluster nodes.

Modern approaches to automation, which are actively used by DevOps engineers in development, make it possible to simplify infrastructure management by implementing automated processes that include building, testing and deploying software. Using unified K8s packages to run standard applications in the container orchestration environment makes the network management process more convenient and predictable for operators.

Currently, 30 virtual machines have been created on the server for laboratory works. All of them are prepared to perform laboratory works. Students deploy Asterisk IP PBXs on these virtual machines. In the future, it is planned to expand the list of laboratory works with the help of additional settings of the Asterisk exchange in the containerized environment.

V. Conclusion

The application of virtualization and containerization technologies to IP telephony is a very timely prospect. The laboratory work described in the article provides great opportunities to study these technologies.

Working with virtual machines and containers, students not only get acquainted with promising virtual IP PBXs, but also gain practical experience of working with such equipment.

Using the Kubernetes platform for Asterisk PBX management allows ensuring flexibility, fault tolerance and scalability of the system, which is an important requirement for modern communication services. The possibilities of process

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on December 01,2025 at 14:43:18 UTC from IEEE Xplore. Restrictions apply.

automation, integration with cloud solutions and network technologies open wide prospects for further development and optimization of such services. This creates the basis for developing and improving the infrastructure of IP telephony systems using a virtual Asterisk PBX in the Kubernetes platform.

REFERENCES

[1] N.N. Vasin, "Packet switching technologies," tutorial. PART 1. Routing and Switching. PGUTI Publ., 2021, 294 p. [in Russian]

[2] V. Olifer, N. Olifer, "Computer Networks: Principles, Technologies and Protocols for Network Design," St. Petersgurg: Piter, 2020, 1008 p. [in Russian]

[3] M.G. Kanischeva, Y.Y. Malikova, I.I. Pelevin and A.P. Pshenichnikov, "The basics of working with the virtual telephone ASTERISK IP-PBX," textbook. Moscow: Media Publisher, 2021, 100 p. [in Russian]

[4] V.M. Antonova, E.E. Malikova, M.S. Stepanov, "Ways of the Asterisk software PBX protection," *T-Comm*, vol. 17, no. 10, 2023, pp. 52-58. DOI: 10.36724/2072-8735-2023-17-10-52-58.

[5] J. Meggelen, L. Madsen, J. Smith, "AsteriskTM: The future of telephony," 2nd ed. Translated from English, St. Petersburg: Symbol-Plus, 2019, 656 p. [in Russian]

[6] A.V. Roslyakov, S.V. Vanyashin, "Future Networks," Samara, PSUTI Publ., 2015, 274 p. [in Russian]

[7] R.L. Smelyanskiy, V.A. Antonenko, "SDN and NFV Principles in Computer Networks," Moscow, KURS, 2020, 160 p. [in Russian]

[8] B.S. Goldshtein, "Infocommunications networks and systems," St. Petersburg, BHV-Petergurg, 2019, 208 p. [in Russian]

[9] V.M. Antonova, V.P. Blinov, M.A. Egorov, A.Y. Malikov, "Studying the Principles of Infocommunication Network Virtualisation Using the Docker Platform," *2024 Systems of Signals Generating and Processing in the Field of on Board Communications*, 2024, pp. 24-28.

[10] A.H. Sayers, I. Miell, "Docker in Practice, Manning," 2019, 384 p.

[11] K8s. Official Site: Container Orchestration Software. URL: https://kubernetes.io/

[12] Asterisk. Official site: PBX Software. URL: https://www.asterisk.org/

[13] S.A. Grushko, A.P. Pshenichnikov, E.E. Malikova and A.Y. Malikov, "Virtual Asterisk IP-PBX Operation Studying and Exploring at the University," *2022 Systems of Signals Generating and Processing in the Field of on Board Communications*, Moscow, Russian Federation, 2022, pp. 15-20.

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on December 01,2025 at 14:43:18 UTC from IEEE Xplore. Restrictions apply.